



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Comunicações por Computador

Ano Letivo de 2025/2026

CC-TP2-PL1-G03

David Silva

a100736

João Costa

ae9271

Tomás Pinto

a104448

Grupo 3

Dezembro, 2025

CC

Índice

1. Introdução	1
2. Arquitetura da Solução	2
2.1. Classe Rover	2
2.2. Topologia Utilizada	3
2.3. Organização dos Programas e Threads	4
2.3.1. Programas da Nave-Mãe	4
2.3.2. Programas do Lado dos Rovers	5
2.3.3. Programa do Ground Control	5
2.3.4. Resumo Estrutural	5
3. Protocolos Aplicacionais	6
3.1. MissionLink (UDP)	6
3.1.1. Arquitetura e Componentes	6
3.1.2. Formato das Mensagens e Estrutura dos Pacotes	8
3.1.3. Funcionamento do Cliente	9
3.1.4. Funcionamento do Servidor	9
3.2. TelemetryStream (TCP)	10
3.2.1. Arquitetura e Componentes	11
3.2.2. Estrutura da Comunicação	11
3.2.3. Formato da Telemetria e Estrutura dos Pacotes	11
3.2.4. Funcionamento do Cliente	12
3.2.5. Funcionamento do Servidor	13
4. API de Observação	15
4.1. Arquitetura da API	15
4.2. Endpoints Disponibilizados	15
4.3. Estrutura das Respostas	16
4.4. Integração com ML e TS	17
4.5. Integração com o Ground Control	17
5. Ground Control	18
5.1. Objetivo e Funcionamento Geral	18
5.2. Comunicação com a API	18
5.3. Interface Gráfica e Atualização do Mapa	18
5.4. Ciclo de Atualização Gráfico	19
5.5. Demonstração do Funcionamento	20
5.6. Justificação das Escolhas de Implementação	20
6. Testes e Resultados	21
6.1. Metodologia de Teste	21
6.2. Teste 1	21
6.3. Teste 2	22
6.4. Teste 3	24
6.5. Discussão dos Resultados	25
7. Conclusão	26

Lista de Figuras

Figura 1	Classe Rover	2
Figura 2	Topologia	3
Figura 3	Diagrama de Sequencia TCP e UDP	6
Figura 4	Interface Grafica GC	19

1. Introdução

A comunicação entre sistemas distribuídos é fundamental em muitas áreas da engenharia informática, especialmente quando é necessário garantir fiabilidade e coordenação entre vários dispositivos. Neste trabalho prático, foi desenvolvido um sistema que simula uma missão espacial composta por uma Nave-Mãe e vários rovers na superfície de um planeta, focando-se na implementação e teste de dois protocolos de comunicação distintos.

O objetivo principal é permitir que a Nave-Mãe envie missões aos rovers e receba deles tanto a telemetria contínua como o progresso da execução. Para isso, foram criados dois módulos de comunicação:

- MissionLink (UDP), responsável pelo envio de missões e recolha de atualizações, exigindo a implementação de mecanismos de fiabilidade ao nível da aplicação;
- TelemetryStream (TCP), responsável pelo envio periódico da telemetria de cada rover.

Além destes protocolos, foi ainda desenvolvida uma API de Observação, que disponibiliza o estado das missões e dos rovers, e um módulo Ground Control, responsável por consumir essa API e mostrar graficamente a evolução da missão.

Todo o sistema foi testado num ambiente CORE com uma topologia que inclui a Nave-Mãe, uma rede de satélites e vários rovers, permitindo avaliar o comportamento da solução em cenários de perda, atraso e concorrência.

2. Arquitetura da Solução

2.1. Classe Rover

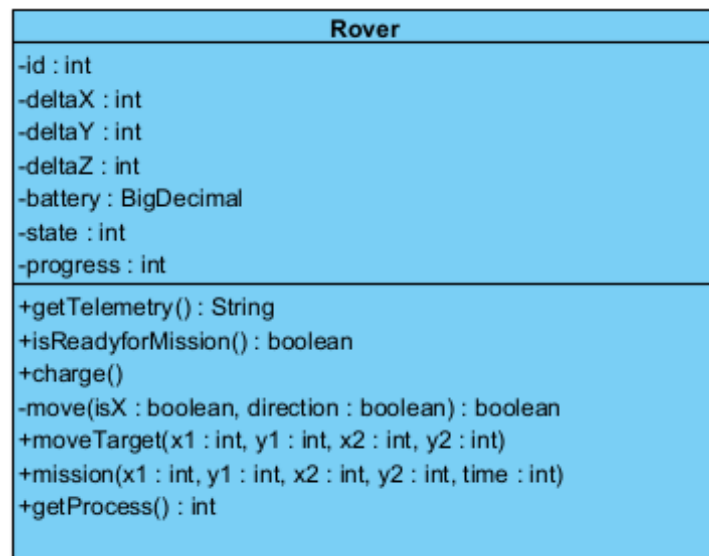


Figura 1: Classe Rover

Como é possível ver na Figura 1, a classe **Rover** representa o agente principal do sistema, responsável por executar missões dentro da área delimitada do mapa. Cada objeto **Rover** é criado com coordenadas (`deltaX`, `deltaY`) e nível de bateria aleatórios, dentro dos limites definidos pelo mapa, enquanto `deltaZ` é fixo, simulando que o rover se move sobre uma planície. A classe fornece um conjunto de métodos que permitem controlar o rover, monitorizar o seu estado e gerenciar suas missões de forma coordenada.

O método **TelemetryStream()** retorna informações essenciais sobre o rover, incluindo o seu identificador (`id`), deslocamento (`deltaX`, `deltaY`, `deltaZ`), nível de bateria (`battery`) e estado (`state`). Para simplificação, o valor de `deltaZ` é sempre constante. O campo `state` indica o estado atual do rover, podendo assumir os seguintes valores: 0 – disponível, 1 – a carregar, 2 – em deslocação, 3 – a executar missão. O método utiliza um **lock** para garantir que as leituras sejam consistentes, evitando que múltiplas threads acessem ao objeto simultaneamente e obtenham valores inconsistentes.

O método **isReadyForMission()** verifica se o rover possui bateria suficiente para alcançar qualquer missão dentro da área definida. Consideramos que o rover consome energia apenas durante a deslocação, movendo-se a uma velocidade constante de 10 m/s e que, se o mesmo tiver pelo menos 50% de bateria, consegue chegar a qualquer missão. Caso a bateria seja insuficiente, o método **charge()** é chamado, carregando a bateria até 100%. O carregamento é incremental e controlado por um lock para manter consistência com o método `getTelemetry()`.

O método **moveTarget()** recebe os quatro vértices de uma missão e escolhe o vértice mais próximo como ponto de partida. O rover move-se em direção a este vértice a 10 m/s, consumindo bateria, já que a energia fornecida pelos painéis solares nessa velocidade não é suficiente para sustentar o movimento. Internamente, este método utiliza o método privado **move()**, que decrementa a bateria a cada metro percorrido.

O método **mission()** simula a execução de uma missão. A partir do vértice em que o rover se encontra, ele percorre a área da missão em **zig-zag**, linha por linha, até completar toda a área ou atingir o limite de tempo definido. Durante esta fase, a velocidade é reduzida para 1 m/s, permitindo que a energia dos painéis solares mantenha a bateria, de modo que o rover não consuma energia. O método também atualiza o campo **progress**, indicando a área percorrida em metros quadrados.

A função **getProgress()** retorna o progresso atual da missão. Se nenhuma missão estiver atribuída, retorna -1. Enquanto o rover se desloca até a missão, o valor é 0. Durante a execução da missão, **progress** representa a área percorrida pelo rover em metros quadrados dentro da região delimitada. Este método também utiliza um lock para garantir consistência com leituras concorrentes.

Em termos de **concorrência**, a classe **Rover** suporta no máximo três threads simultâneas:

- **T1**: chamada de **getTelemetry()**;
- **T2**: chamadas de **isReadyForMission()**, **charge()**, **moveTarget()** e **mission()**;
- **T3**: chamada de **getProgress()**.

Cada programa **MainRover** instancia o seu próprio objeto **Rover**, garantindo que cada agente opere de forma independente. O uso de **ReentrantLock** em métodos críticos garante que leituras e escritas não causem inconsistências, especialmente quando múltiplas threads interagem com o mesmo objeto.

2.2. Topologia Utilizada

A topologia implementada no ambiente de emulação **CORE**, representada na Figura 2, foi desenvolvida para modelar o cenário espacial descrito no enunciado: uma Nave-Mãe responsável pela gestão das operações, vários rovers distribuídos na superfície do planeta e uma rede intermédia de satélites que assegura o encaminhamento multi-salto entre todos os nós. Esta arquitetura cumpre os requisitos mínimos definidos para o trabalho e proporciona um ambiente realista para testar os protocolos **MissionLink** (UDP) e **TelemetryStream** (TCP).

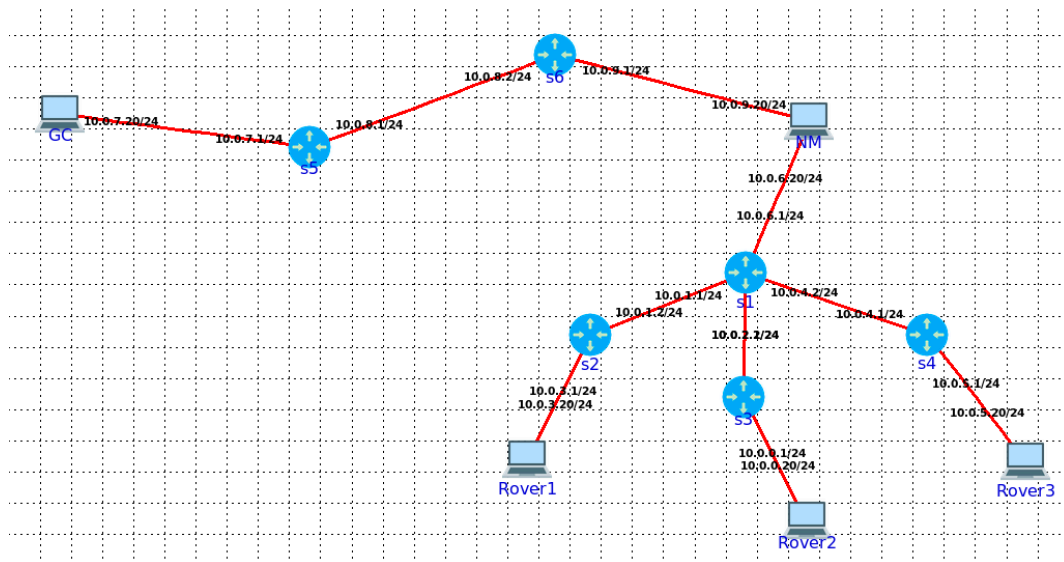


Figura 2: Topologia

A solução é composta por quatro grupos principais de elementos:

1. **Ground Control (GC)** – Localizado no extremo esquerdo da topologia, este nó liga-se ao router **S5** e atua como cliente da API de Observação exposta pela Nave-Mãe. É responsável por apresentar ao utilizador o estado das missões e a telemetria recebida dos rovers, simulando um centro de controlo terrestre.
2. **Nave-Mãe (NM)** – Posicionada à direita da topologia, a Nave-Mãe liga-se diretamente ao router **S1** e assume o papel central do sistema. Funciona como:
 - servidor UDP do **MissionLink**, encarregue da atribuição de missões;

- servidor TCP do TelemetryStream, responsável por receber telemetria contínua;
- servidor HTTP para disponibilização da API de Observação.

A sua localização no topo lógico da hierarquia força os rovers a comunicar através de múltiplos saltos, refletindo as condições próprias das comunicações espaciais.

Além disso, a implementação da Nave-Mãe foi dividida em dois programas distintos. O primeiro, *MainNaveGC*, é responsável por disponibilizar a API de Observação e comunicar com o Ground Control. O segundo, *MainNaveR*, contém os módulos de comunicação com os rovers, integrando simultaneamente o servidor UDP do MissionLink e o servidor TCP do TelemetryStream. Esta separação permite isolar responsabilidades e garantir que a interface externa permanece independente da lógica de comunicação com os rovers.

3. **Rede de Satélites (routers S1 a S6)** – Estes routers constituem o backbone da arquitetura, assegurando o encaminhamento entre o Ground Control, a Nave-Mãe e os rovers.
4. **Rovers (Rover1, Rover2 e Rover3)** – Distribuídos na periferia inferior da topologia, cada um ligado a um router distinto (S2, S3 ou S4). Os rovers atuam como clientes dos dois protocolos aplicacionais: solicitam missões via MissionLink (UDP) e enviam telemetria periódica através do TelemetryStream (TCP).

Em conjunto, esta arquitetura fornece uma representação clara e funcional das comunicações num ambiente espacial simulado, permitindo avaliar a robustez dos protocolos desenvolvidos, o desempenho sob condições adversas e o comportamento da Nave-Mãe como ponto central de gestão e observação.

2.3. Organização dos Programas e Threads

Esta secção descreve a estrutura geral das threads da solução. O comportamento específico das threads do TelemetryStream (TCP) e do MissionLink (UDP) é detalhado nas respetivas secções.

A solução completa é constituída por quatro programas principais, cada um responsável por um conjunto distinto de funcionalidades dentro do sistema. É importante notar que os ficheiros presentes nas pastas /udp e /tcp não são programas autónomos: representam módulos que são executados em threads internas criadas pelos programas principais.

2.3.1. Programas da Nave-Mãe

A Nave-Mãe é dividida logicamente em dois processos independentes:

MainNaveR Responsável pela comunicação direta com os rovers. Ao iniciar, cria duas threads principais:

- ServerTS — servidor TCP do TelemetryStream; encarregue de receber telemetria contínua de todos os rovers.
- ServerML — servidor UDP do MissionLink; responsável por receber pedidos de missão, updates e partes de ficheiros.

Cada um destes servidores cria threads internas adicionais:

- ServerTS cria uma nova thread por cada ligação TCP estabelecida por um rover.
- ServerML cria um RoverHandlerML dedicado a cada rover, gerindo filas de pacotes e acknowledgements.

Esta estrutura permite que a Nave-Mãe comunique simultaneamente com vários rovers, mantendo isolamento e concorrência segura entre streams.

MainNaveGC Este programa disponibiliza apenas a API de Observação, usando um servidor HTTP Spark Java. Não interage diretamente com os rovers, mas expõe endpoints HTTP que consultam os ficheiros produzidos pelo MissionLink e TelemetryStream.

2.3.2. Programas do Lado dos Rovers

MainRover Simula um rover individual. Ao iniciar, cria duas threads principais:

- **ClientTS** — cliente TCP que envia telemetria em cadência fixa (1 segundo).
- **ClientML** — cliente UDP responsável pela solicitação e execução de missões.

Estas threads funcionam de forma totalmente independente: enquanto o rover executa a missão (numa thread criada pelo **ClientML**), a telemetria continua a ser enviada sem interrupções.

2.3.3. Programa do Ground Control

MainGC Atua como consumidor externo da API disponibilizada pela Nave-Mãe. Este programa executa um ciclo periódico que, a cada segundo:

- envia pedidos HTTP aos endpoints `/rovers` e `/missões`,
- atualiza o mapa visual do cenário com a informação recebida.

O Ground Control não participa nos protocolos **MissionLink** ou **TelemetryStream**, garantindo total desacoplamento entre observação e execução.

2.3.4. Resumo Estrutural

A organização final das threads é a seguinte:

- Cada rover:
 - 2 threads principais (**ClientTS**, **ClientML**)
 - 1 thread extra sempre que uma missão está a ser executada
- Nave-Mãe (Rovers):
 - 2 threads principais (**ServerTS**, **ServerML**)
 - 1 thread adicional por ligação TCP (telemetria)
 - 1 thread **RoverHandlerML** por rover (UDP)
- Nave-Mãe (Ground Control):
 - Usa API **Spark Java** que disponibiliza um servidor HTTP
- Ground Control:
 - 1 thread principal com ciclo de atualização periódica

Esta divisão proporciona concorrência estável, isolamento entre componentes e escalabilidade natural com o número de rovers.

3. Protocolos Aplicacionais

A Figura 3 apresenta um diagrama de sequência que ilustra a comunicação entre a Nave-Mãe (MainNaveR) e um Rover (MainRover). O diagrama mostra de forma clara os dois protocolos aplicacionais utilizados: MissionLink (UDP), responsável pelo envio de missões e atualização de progresso, e TelemetryStream (TCP), encarregado da transmissão contínua da telemetria. Também são representadas as duas threads do rover: uma dedicada à execução da missão e outra ao envio das telemetrias, permitindo observar a interação concorrente entre os módulos e a Nave-Mãe.

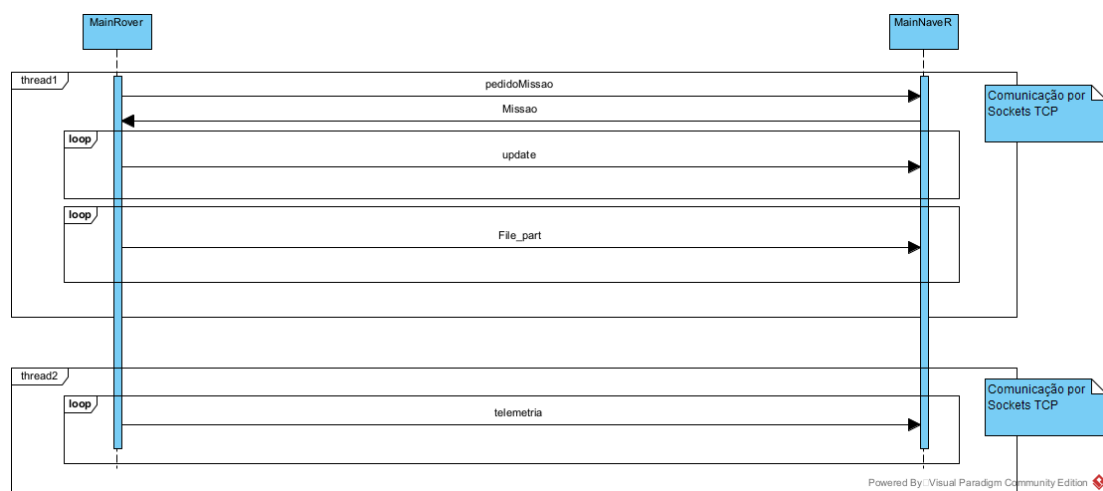


Figura 3: Diagrama de Sequencia TCP e UDP

3.1. MissionLink (UDP)

O módulo MissionLink implementa o subsistema responsável pela comunicação em modo datagrama entre a Nave e os Rovers, sendo utilizado para a transmissão de ordens de missão e para a recolha periódica de progresso das operações no terreno. A comunicação é realizada através do protocolo UDP e a sua utilização revela-se tecnicamente adequada ao papel desempenhado pelo MissionLink, dada a necessidade de comunicações leves, rápidas e orientadas a mensagens, sem a sobrecarga associada a mecanismos de conexão.

Ao contrário do TelemetryStream (TCP), que privilegia um fluxo contínuo de dados fiável, o MissionLink opera num paradigma distinto: o objetivo é enviar mensagens autónomas (missões e acknowledgements), onde a latência reduzida e a simplicidade de processamento são fatores relevantes. Como UDP não garante entrega, ordem ou ausência de duplicação, foi necessário implementar neste módulo um conjunto de mecanismos de controlo e fiabilidade aplicacional, detalhados mais à frente neste capítulo.

3.1.1. Arquitetura e Componentes

O MissionLink segue um modelo cliente-servidor baseado em UDP, suportado por um conjunto de mecanismos aplicacionais que asseguram fiabilidade, controlo de duplicados e execução concorrente.

Lado do Cliente (Rover)

O cliente (ClientML) é executado numa thread principal dedicada ao MissionLink.

Esta thread:

- Estabelece um socket UDP efêmero para comunicar com a Nave-Mãe.
- Periodicamente envia um pedido de missão (PEDIR_MISSAO).
- Caso receba uma missão válida, inicia uma thread separada que executa a missão através do objeto Rover, permitindo que a comunicação e a execução da missão ocorram em paralelo.
- Implementa um mecanismo de retransmissão com limite de tentativas para todos os pacotes que exigem confirmação (ACK).
- Reenvia automaticamente um ACK caso receba novamente uma missão que já se encontra em execução.
- Ignora pacotes inesperados, evitando alterações indevidas no estado interno.
- No fim da missão, envia o ficheiro final fragmentado em várias partes (FILE_PART), cada uma delas sujeita a retransmissão caso não seja confirmado o respetivo ACK.

Este comportamento segue um modelo **stop-and-wait**: cada pacote enviado contém toda a informação necessária para o recetor processar isoladamente, e o cliente aguarda sempre o ACK correspondente antes de prosseguir para o envio seguinte.

Lado do Servidor (Nave-Mãe)

O servidor possui uma thread principal que escuta numa porta UDP fixa e recebe todas as comunicações provenientes dos rovers.

Sempre que recebe um pacote:

- Identifica o rover de origem.
- Cria (na primeira comunicação) ou reutiliza o respetivo RoverHandlerML.
- Coloca o pacote na fila interna do handler correspondente.

Cada RoverHandlerML tem:

- Uma thread dedicada que processa exclusivamente um rover específico.
- Duas filas internas:
 - **FilaPacotes** para pacotes regulares (PEDIR_MISSAO, UPDATE, FILE_PART).
 - **FilaAcks** para ACKs recebidos.
- Um conjunto de missões únicas atribuídas apenas a esse rover.
- Estruturas auxiliares para controlo de duplicados, tanto para UPDATE como para FILE_PART.
- Um socket próprio para envio de pacotes ao rover.

A sincronização entre a thread principal do servidor e os handlers é feita com mecanismos de sinalização baseados em locks e conditions. Os detalhes do ciclo interno de processamento são aprofundados na secção seguinte “Funcionamento do Servidor”.

Sobre o Conceito de Timeout

O timeout é implementado de duas formas distintas:

1. **Timeout nativo do socket** Utilizado quando se faz `socket.receive()`.
2. **Replicação da lógica de timeout** Usada, por exemplo, na fila de ACKs do RoverHandlerML, onde não existe espera diretamente no socket, mas sim num mecanismo interno que imita o comportamento do timeout de um socket.

Em ambos os casos, a regra de funcionamento é idêntica: **Sempre que qualquer pacote chega, mesmo que não seja o esperado, o contador do timeout é reiniciado.**

Exemplo: Se se está a aguardar um ACK durante 2 segundos e chega um pacote inesperado ao fim de 1.5 segundos, o pacote é ignorado, mas o temporizador é reiniciado, passando a ser necessário esperar mais 2 segundos completos até ocorrer timeout real.

3.1.2. Formato das Mensagens e Estrutura dos Pacotes

O **MissionLink** implementa uma estratégia de comunicação stop-and-wait. Cada pacote transporta toda a informação necessária para ser processado de forma independente, e o emissor só envia o pacote seguinte após receber o respetivo ACK. Esta abordagem simplifica o controlo de fluxo, facilita a deteção de duplicados e garante fiabilidade mesmo sobre UDP.

A comunicação é baseada em mensagens UDP com payload textual. Cada tipo de mensagem segue um formato específico para permitir interpretação imediata pelo servidor:

- **PEDIR_MISSAO** (Rover -> Nave Mãe)

Campo	MISSAO (0x0)
Tamanho (bytes)	1

Mensagem simples: "PEDIR_MISSAO" O cliente envia este pacote para solicitar a próxima missão disponível.

- **ENVIO_MISSAO** (Nave Mãe -> Rover)

Campo	MISSAO (0x01)	ID	x1	y1	x2	y2	max_tempo	intervalo	tamanho	descrição
Tamanho (bytes)	1	4	4	4	4	4	4	4	4	Variável

Contém todos os parâmetros necessários para o rover executar a missão, incluindo coordenadas, descrição, limite máximo e intervalo de atualização.

- **ACK_MISSÃO** (Rover -> Nave Mãe)

Campo	ACK (0x04)	Tipo ACK	ID
Tamanho (bytes)	1	1	4

Confirma receção de MISSAO. Permite ao cliente retransmitir pacotes apenas quando necessário.

- **UPDATE** (Rover -> Nave Mãe)

Campo	UPDATE (0x02)	ID	SEQ	Progresso
Tamanho (bytes)	1	4	4	4

Informa o progresso atual da missão. Inclui um número de sequência (SEQ) para permitir deteção de duplicados e ACK correspondente.

- **ACK_UPDATE** (Nave Mãe -> Rover)

Campo	ACK (0x04)	Tipo ACK	ID	SEQ
Tamanho (bytes)	1	1	4	4

Confirma receção de UPDATE. Permite ao cliente retransmitir pacotes apenas quando necessário.

- **FILE_PART** (Rover -> Nave Mãe)

Campo	FILE_PART (0x04)	ID	SEQ	Tamanho	Dados
Tamanho (bytes)	1	4	4	4	Variável

Fragmenta o ficheiro final da missão em partes de tamanho fixo (exceto a última), permitindo envio seguro via UDP. Cada parte é confirmada com ACK antes de enviar a seguinte.

- **ACK_FILE_PART** (Nave Mãe -> Rover)

Campo	ACK (0x04)	Tipo ACK	ID	SEQ
Tamanho (bytes)	1	1	4	4

Confirma receção de FILE_PART. Permite ao cliente retransmitir pacotes apenas quando necessário.

Características do formato de mensagens UDP:

- Mensagens independentes, sem necessidade de separar tamanho de payload, já que cada pacote é completo.
- Textual, legível e facilmente interpretável por ferramentas externas.
- Estrutura consistente que permite deteção de duplicados, ordenação e validação.
- Fiabilidade assegurada pelo esquema de **ACK e retransmissão**, compensando a natureza não fiável do UDP.

Esta abordagem garante que, mesmo com perda ou atraso de pacotes, o servidor e o rover mantêm **consistência e integridade da missão**, enquanto a implementação continua simples e eficiente.

3.1.3. Funcionamento do Cliente

O ciclo de operação do rover é o seguinte:

1. Verifica se a bateria é suficiente; caso contrário, recarrega.
2. Envia um pedido de missão (PEDIR_MISSAO) para a Nave Mãe.
3. Aguarda a receção da missão. Em caso de timeout, reenvia o pedido.
4. Recebe a missão e envia o ACK correspondente.
5. Inicia a execução da missão numa thread separada, permitindo que comunicação e execução ocorram em paralelo.
6. Durante a missão, envia periodicamente pacotes UPDATE (1 byte + ID + SEQ + PROGRESSO), aguardando o ACK correspondente antes de enviar o seguinte.
7. Ao concluir a missão, envia o ficheiro final (FILE_PART) dividido em partes, aguardando ACK de cada parte antes de enviar a próxima.
8. Repete o ciclo para a missão seguinte.

Decisões de implementação importantes:

- Retransmissão com timeout e número máximo de tentativas garante fiabilidade sobre UDP.
- Threads separadas evitam bloqueios, permitindo comunicação contínua enquanto a missão é executada.
- Registo de pacotes recebidos evita reprocessamento de duplicados e mantém consistência nos ficheiros finais.
- Todas as mensagens seguem o modelo stop-and-wait, contendo informação suficiente para processamento independente pelo servidor: PEDIR_MISSAO, ACK_MISSÃO, UPDATE, ACK_UPDATE, FILE_PART, ACK_FILE_PART.

3.1.4. Funcionamento do Servidor

O servidor MissionLink possui uma thread principal que escuta continuamente numa porta UDP fixa.

Sempre que um pacote chega:

- a thread principal identifica o rover de origem através do endereço e porta do datagrama;
- cria o RoverHandlerML correspondente (caso ainda não exista);
- e coloca o pacote recebido na fila interna desse handler.

Só depois desta etapa é que o processamento especializado é realizado dentro da thread dedicada de cada rover.

O servidor MissionLink é responsável por gerir simultaneamente múltiplos rovers e assegurar a entrega correta das missões, atualizações e ficheiros finais. Para isso, cada rover conectado é associado a um

RoverHandlerML, executado numa thread dedicada, garantindo isolamento entre clientes e paralelismo seguro.

Cada RoverHandlerML não se limita a processar os pacotes que recebe. Além da fila de pacotes (filaPacotes), o handler mantém também uma fila de ACKs (filaACKs), utilizada para confirmar as mensagens que ele próprio envia ao rover, como missões, ACKs de atualização e ACKs de ficheiros.

Quando o handler envia uma missão ou um ACK, o método enviarComACK() entra num ciclo de espera onde verifica periodicamente se o ACK correspondente foi colocado na filaACKs. Esta fila é preenchida pelo ServerML, que entrega ao handler todos os pacotes recebidos do rover. Assim, o handler consegue:

- aguardar a confirmação de cada pacote enviado,
- detetar retransmissões necessárias,
- filtrar ACKs inesperados,
- e garantir fiabilidade sobre UDP.

Este mecanismo de duplo fluxo, fila de pacotes recebidos + fila de ACKs de confirmação, assegura que cada RoverHandlerML controla, de forma independente e concorrente, a comunicação fiável com o rover associado.

O fluxo de processamento dos pacotes segue estas regras:

1. Pedido de missão (**PEDIR_MISSAO**)
 - O servidor identifica o rover e verifica quais missões estão disponíveis.
 - A primeira missão não atribuída é selecionada e enviada ao rover.
 - É enviado um ACK de confirmação, garantindo que o rover sabe que a missão foi recebida.
 - A missão só é removida da fila interna após a confirmação, evitando perda em caso de retransmissões.
2. Atualizações de progresso (**UPDATE**)
 - Cada pacote contém o número sequencial e o progresso da missão.
 - O servidor mantém um registo de updates recebidos por missão, evitando processar duplicados.
 - As atualizações válidas são escritas em updates_missao<ID>.log numa posição fixa, usando *RandomAccessFile* com *FileLock* para permitir escrita concorrente segura.
 - É enviado um ACK para cada update confirmado.
3. Partes de ficheiro final (**FILE_PART**)
 - Cada parte é gravada em dados_missao<ID>.log na posição correta, calculada a partir do índice da parte.
 - Falhas no ficheiro são preenchidos com caracteres de placeholder ('-') para manter consistência.
 - Pacotes repetidos são ignorados para evitar sobreposição de dados.
 - Cada parte confirmada é respondida com um ACK.

Para suportar esta operação de forma segura e eficiente:

- **Concorrência:** o acesso às filas de pacotes e ACKs é sincronizado usando *ReentrantLock* e *Condition*, evitando condições de corrida e permitindo que threads aguardem novos pacotes sem consumir CPU.
- **Preservação da ordem:** as missões são armazenadas em *LinkedHashMap*, garantindo que cada rover recebe missões na ordem prevista pelo planeamento da missão.
- **Robustez:** a combinação de retransmissão, verificação de duplicados e ACKs torna o sistema tolerante a perdas e duplicações típicas de UDP, mantendo a integridade dos dados recebidos.

3.2. TelemetryStream (TCP)

As threads utilizadas neste módulo seguem a organização geral apresentada na secção anterior, sendo aqui descrito o funcionamento concreto do canal TCP.

O módulo **TelemetryStream** foi desenvolvido para assegurar a transmissão fiável da telemetria gerada pelos rovers para a Estação de Controlo. Ao contrário do MissionLink (UDP), que privilegia rapidez e tolerância a perdas, o **TelemetryStream** utiliza TCP para garantir a integridade, entrega e ordenação dos dados provenientes dos sensores internos e do estado operacional dos rovers.

É utilizado o TCP nesta componente pois a telemetria tem natureza sequencial e crítica, devendo ser armazenada na ordem correta sem perdas ou duplicações. Esta informação é essencial para auditoria, reconstrução de trajetórias e detecção de anomalias durante as missões.

3.2.1. Arquitetura e Componentes

O `TelemetryStream` segue um modelo cliente–servidor persistente:

- O rover atua como cliente TCP, estabelecendo uma ligação estável à Nave Mãe.
- A Nave Mãe atua como servidor TCP, capaz de receber múltiplos rovers simultaneamente.
- Cada ligação é tratada numa thread dedicada, isolando falhas e garantindo paralelismo.
- A telemetria é enviada de segundo a segundo, num formato binário compacto.

Este desenho arquitetural permite um fluxo contínuo, fiável e escalável, sem afetar o funcionamento de outros módulos da aplicação.

3.2.2. Estrutura da Comunicação

Ciclo de funcionamento do Cliente

O cliente executa o seguinte ciclo:

1. Abre uma ligação **TCP** para o servidor.
2. Obtém a telemetria em formato *JSON* interno:
3. *String mensagem = rover.getTelemetry();*
4. Extraí manualmente cada campo (inteiros e bytes).
5. Serializa os dados para um pacote binário de tamanho fixo, utilizando um *ByteBuffer*.
6. Envia primeiro o tamanho do pacote (int de 4 bytes).
7. Envia o pacote binário propriamente dito.
8. Aguarda 1 segundo e repete.

A decisão de enviar o tamanho antes do pacote é fundamental, porque TCP é um stream contínuo sem fronteiras de mensagem. Sem este mecanismo, o servidor não saberia onde um pacote termina e o seguinte começa.

Ciclo de funcionamento do Servidor

Para cada cliente, o servidor:

1. Lê o tamanho do próximo pacote via *DataInputStream.readInt()*.
2. Aloca um array do tamanho apropriado.
3. Lê exatamente N bytes utilizando *readFully()*, garantindo que nenhum pacote é fragmentado.
4. Reconstrói os campos através de *ByteBuffer.wrap()*.
5. Constrói novamente uma representação JSON para persistência.
6. Anexa o registo ao ficheiro de telemetria do rover.

Cada ligação é atendida numa thread dedicada, permitindo suporte concorrente para múltiplos rovers.

3.2.3. Formato da Telemetria e Estrutura dos Pacotes

A telemetria gerada pelo modelo do rover é inicialmente produzida como uma string JSON. No entanto, a transmissão direta em JSON implicaria:

- maior overhead (strings mais extensas),
- parsing mais pesado do lado do servidor,
- dependência de bibliotecas externas de serialização,

- variabilidade de tamanho que complicaria a reconstrução no stream TCP.

Para evitar estes inconvenientes, o **TelemetryStream** opta por converter a telemetria num pacote binário fixo de 21 bytes, através de um `ByteBuffer`. Esta abordagem oferece:

- menor tamanho e maior eficiência,
- estrutura mais simples de ler e reconstruir,
- ausência de ambiguidade na interpretação dos tipos,
- excelente desempenho para envio periódico.

Tabela do Pacote Binário

Campo	Tipo	Tamanho
id	int	4 bytes
deltaX	int	4 bytes
deltaY	int	4 bytes
deltaZ	int	4 bytes
battery	byte	1 byte
state	byte	1 byte
erroPosicao	byte	1 byte
erroBateria	byte	1 byte
erroEstado	byte	1 byte
Total	— —	21 bytes

Envio do tamanho do pacote

Antes do envio do pacote, o cliente transmite 4 bytes contendo o comprimento total do pacote (21). Isto acontece para:

- Permitir ao servidor reconstruir exatamente cada pacote.
- Remover dependências de delimitadores textuais.
- Garantir que o `readFully()` trabalha sobre os limites corretos.

3.2.4. Funcionamento do Cliente

O módulo **ClientTS** é responsável por estabelecer a ligação TCP ao servidor e transmitir periodicamente a telemetria gerada pelo modelo interno do rover. A ligação é criada uma única vez e mantida ativa durante todo o ciclo de operação, garantindo um fluxo estável e contínuo de dados.

Ligação TCP persistente

A implementação opta por uma ligação **persistente**, evitando abrir e fechar sockets a cada envio, esta abordagem apresenta vantagens claras:

- redução significativa do overhead associado ao handshake TCP,
- latência muito inferior, assegurando entrega imediata da telemetria,
- manutenção automática da ordenação dos pacotes, fundamental para análises temporais,
- deteção imediata de falhas, dado que qualquer quebra no socket resulta em *exception*.

A decisão é particularmente adequada porque o rover envia telemetria em cadência fixa e previsível, sem interações intermitentes ou baseadas em pedidos.

Cadência de envio fixa (1 segundo)

A transmissão é executada num ciclo contínuo, com intervalos definidos por:

```
Thread.sleep(1000);
```

Isto estabelece rigorosamente uma cadência de 1 segundo por pacote, garantindo regularidade temporal e facilitando a análise posterior.

Parsing minimalista do JSON interno

Embora a telemetria seja inicialmente produzida em JSON, a conversão para o formato binário requer apenas a extração de valores numéricos. Para evitar dependências externas e preservar a simplicidade do ambiente embarcado, optou-se por implementar métodos dedicados (`extrairInt` e `extrairByte`).

Com esta abordagem conseguimos algumas vantagens:

- remoção de bibliotecas externas,
- parsing mais leve e eficiente,
- comportamento tolerante a falhas simples (valores inválidos devolvem 0, sem interromper o ciclo).

Envio no formato [tamanho + bloco de dados]

O envio é estruturado como:

```
[int length][byte[] data]
```

Esta decisão é essencial porque o TCP não delimita mensagens — trata todo o fluxo como uma sequência contínua de bytes. Se o cliente enviasse apenas o bloco binário, o servidor não teria forma de saber onde cada pacote termina.

Ao enviar primeiro o tamanho:

- o servidor consegue reconstruir o pacote de forma exata,
- evitam-se problemas de fragmentação (pacotes divididos em múltiplos reads),
- e também de concatenação (vários pacotes lidos de uma só vez).

A escolha é, portanto, diretamente motivada pela natureza do TCP e garante previsibilidade na receção.

3.2.5. Funcionamento do Servidor

O módulo `ServerTS` recebe a telemetria dos rovers, reconstrói os pacotes binários e regista a informação num ficheiro individual por rover. O servidor opera de forma concorrente, permitindo a ligação simultânea de múltiplos emissores.

Arquitura multithreaded

Cada ligação recebida através de `ServerSocket.accept()` originou uma thread independente, responsável por:

1. ler o tamanho do pacote com `readInt()`,
2. ler exatamente o número indicado de bytes,
3. reconstruir os campos através de um `ByteBuffer`,
4. converter os valores extraídos de volta para JSON,
5. registar a telemetria no ficheiro correspondente.

Armazenamento estruturado da telemetria

Cada rover dispõe do seu próprio ficheiro:

```
telemetria_rover<ID>.log
```


Esta organização permite:

- revisão individual do comportamento dos rovers,
- análises simples e diretas,
- persistência imediata sem dependência de bases de dados,
- escrita eficiente graças ao recurso a *BufferedWriter*.

O servidor escreve cada entrada de telemetria no formato JSON, preservando a estrutura original e facilitando o consumo por ferramentas externas.

Deteção fiável de desconexões

A leitura do tamanho do pacote utiliza *readInt()*. Se o cliente encerrar a ligação, o fluxo gera *EOFException*, permitindo:

- identificação explícita do fim da ligação,
- libertação imediata da thread associada,
- prevenção de fugas de recursos.

Esta abordagem torna o comportamento do servidor previsível e robusto face a falhas ou desconexões normais.

4. API de Observação

A API de Observação constitui o componente responsável por disponibilizar, de forma estruturada e externa, toda a informação relevante sobre o estado da missão e dos rovers. Este módulo permite que o nó Ground Control aceda, em tempo real ou quase real, ao progresso das missões, à telemetria mais recente dos rovers e ao estado atual do sistema. A API é executada na Nave-Mãe e funciona como camada de integração entre os protocolos aplicacionais (MissionLink e TelemetryStream) e o utilizador final.

A implementação foi realizada utilizando o microframework SparkJava, suportado por Jetty, permitindo criar rapidamente endpoints HTTP leves e eficientes. A serialização dos dados é efetuada com recurso a Gson, convertendo automaticamente objetos Java em JSON legível

4.1. Arquitetura da API

A API é executada no processo principal da Nave-Mãe, sendo inicializada pelo programa **MainNaveGC**.

Os principais passos efetuados na inicialização são:

1. Configuração da interface de rede e porta de escuta:

```
ipAddress(ip);  
port(portNumber);
```

2. Instanciação global do objeto Gson para serialização JSON.
3. Registo dos endpoints HTTP, cada um mapeado para um handler que recolhe e transforma os dados internos do sistema para exposição externa.

A API é exclusivamente de leitura: nenhum endpoint altera o estado das missões ou dos rovers.

O estado é produzido pelo MissionLink (UDP) e TelemetryStream (TCP) e apenas consumido pela API.

4.2. Endpoints Disponibilizados

A API disponibiliza dois endpoints principais:

GET /rovers

Este endpoint devolve a telemetria mais recente disponível para cada rover ativo.

O handler correspondente:

```
get("/rovers", (req, res) -> {  
    res.type("application/json");  
    return gson.toJson(RoverService.readAllRovers());  
});
```

O método *RoverService.readAllRovers()* percorre todos os ficheiros gerados pelo TelemetryStream:

```
telemetria_rover<ID>.log
```

Para cada ficheiro vai obter a última linha, ou seja, a telemetria mais recente, reconstrói um objeto *RoverInfo* e adiciona-o à lista final.

Cada objeto devolvido vai possuir os campos id, deltaX, deltaY, deltaZ, battery, state e os marcadores de erro (erroPosicao, erroBateria, erroEstado)

Este formato permite ao Ground Control identificar anomalias e acompanhar a evolução dos rovers.

GET /missoes

Este endpoint expõe todas as missões existentes, juntamente com o respetivo estado de progresso:

```
get("/missoes", (req, res) -> {  
  res.type("application/json");  
  return gson.toJson(MissionService.readMissions());  
});
```

O método *MissionService.readMissions()* lê o ficheiro principal:

```
missoes.txt
```

Cada linha desse ficheiro contém, em JSON, uma instância *MissionInfo* com tipo, id, coordenadas (x1,y1,x2,y2), tempo máximo, intervalo de tempo, tamanho e descrição.

Após reconstrução da missão, o serviço calcula dinamicamente o estado atual do progresso, consultando:

- dados_missao<ID>.log → missão concluída (progresso = -2)
- updates_missao<ID>.log → última atualização enviada pelo rover
- caso não exista progresso → progresso = -1

Esta abordagem garante que o estado devolvido reflete sempre a execução real dos rovers, independentemente de falhas na comunicação.

4.3. Estrutura das Respostas

As respostas são devolvidas em JSON, produzidas automaticamente por Gson.

Exemplo de um endpoint /rovers:

```
[  
  {  
    "id": 1,  
    "deltaX": 57,  
    "deltaY": 12,  
    "deltaZ": 0,  
    "battery": 82,  
    "state": 2,  
    "erroPosicao": 0,  
    "erroBateria": 0,  
    "erroEstado": 0  
  }  
]
```

E do endpoint /missoes:

```
[
  {
    "tipo": "MISSAO",
    "id": 3,
    "x1": -200,
    "y1": 200,
    "x2": -170,
    "y2": 210,
    "descricao": "Medição de temperatura do solo",
    "tempo": 180,
    "intervalo": 4,
    "progresso": 55
  }
]
```

4.4. Integração com ML e TS

A API de Observação integra-se de forma indireta com os restantes módulos da solução, funcionando essencialmente como uma camada de leitura sobre os ficheiros produzidos pelos protocolos MissionLink (UDP) e TelemetryStream (TCP). Em vez de aceder diretamente às estruturas internas de cada módulo ou manter estados em memória, a API baseia-se exclusivamente nos registos persistidos no sistema de ficheiros, o que garante uma separação clara entre a lógica de comunicação e a disponibilização dos dados ao exterior.

No caso do MissionLink, são gerados ficheiros do tipo *updates_missao<ID>.log* e *dados_missao<ID>.log*, que armazenam, respetivamente, as atualizações de progresso e a conclusão final de cada missão. Por outro lado, o TelemetryStream produz ficheiros *telemetria_rover<ID>.log*, onde é escrita continuamente a telemetria enviada pelos rovers. A API consulta estes ficheiros sempre que recebe um pedido, determinando assim, em tempo real, o estado mais recente do sistema.

Este modelo de integração proporciona vários benefícios. Em primeiro lugar, garante um elevado nível de isolamento entre os diferentes componentes, evitando dependências diretas e facilitando a manutenção. Em segundo lugar, o uso de *FileLocks* assegura segurança concorrente, prevenindo leituras inconsistentes quando múltiplos processos acedem simultaneamente aos mesmos ficheiros. Para além disso, a utilização de ficheiros como mecanismo de persistência permite que toda a informação seja preservada mesmo em caso de falhas ou reinícios inesperados dos processos, assegurando maior robustez ao sistema. Finalmente, esta abordagem permite ainda a realização de análises posteriores, uma vez que todos os eventos relevantes ficam registados de forma permanente.

4.5. Integração com o Ground Control

O módulo Ground Control consome os endpoints através de pedidos HTTP simples. Estes dados são utilizados para:

- apresentar o estado atual de cada rover,
- visualizar o progresso das missões,
- detetar anomalias ou atrasos,
- sincronizar a interface com a evolução do sistema.

A API foi projetada para ser estável, consistente e facilmente integrável em aplicações externas, assegurando o papel central da Nave-Mãe enquanto ponto agregador de toda a informação.

5. Ground Control

O módulo Ground Control (GC) funciona como o ponto de observação externo do sistema, permitindo monitorizar graficamente, em tempo real, o estado das missões e a telemetria dos rovers. O GC consome a API de Observação disponibilizada pela Nave-Mãe e atualiza automaticamente um mapa visual do cenário, simulando o ambiente operacional de um centro de controlo espacial.

5.1. Objetivo e Funcionamento Geral

O GC atua exclusivamente como cliente HTTP da API da Nave-Mãe. Não participa nos protocolos MissionLink (UDP) nem TelemetryStream (TCP); em vez disso, recolhe apenas a informação consolidada que estes módulos produzem.

A aplicação recebe o endereço IP e a porta do servidor de API e constrói a base URL para todas as requisições subsequentes. Toda a lógica de atualização se baseia num ciclo periódico que consulta a API a cada segundo.

5.2. Comunicação com a API

O GC realiza duas requisições essenciais a cada iteração do ciclo:

- GET /rovers – obtém a telemetria mais recente de cada rover, incluindo posição, bateria e estado.
- GET /missoes – recolhe o estado da missão: descrição, progresso, limites espaciais e estado atual.

A implementação utiliza HttpURLConnection, garantindo:

- robustez perante falhas temporárias,
- leitura automática do errorStream em caso de erro HTTP,
- baixo overhead e simplicidade.

5.3. Interface Gráfica e Atualização do Mapa

O Ground Control apresenta uma interface gráfica completa, dividida em três zonas principais:

1. Mapa das Missões

O painel central representa os limites do mapa em grelha uniforme. Nele são desenhados:

- as **missões** como retângulos identificados por “M1”, “M2”, ...;
- os **rovers** como um ponto colorido em movimento contínuo;
- a **missão atribuída** a cada um dos rovers, destacada com cor diferente.

2. Tabela de Rovers (topo direito)

Lista os rovers ativos e apresenta:

- ID
- posição atual (deltaX, deltaY)
- percentagem de bateria
- estado

3. Tabela de Missões (base direita)

Apresenta, para todas as missões:

- ID
- descrição
- estado da missão
- progresso (%)

A Figura 4 apresenta um exemplo da interface gráfica onde podemos ver 3 rovers, o R1 a deslocar-se para a missão M4, o R2 a carregar e o R3 a executar a missão M1.

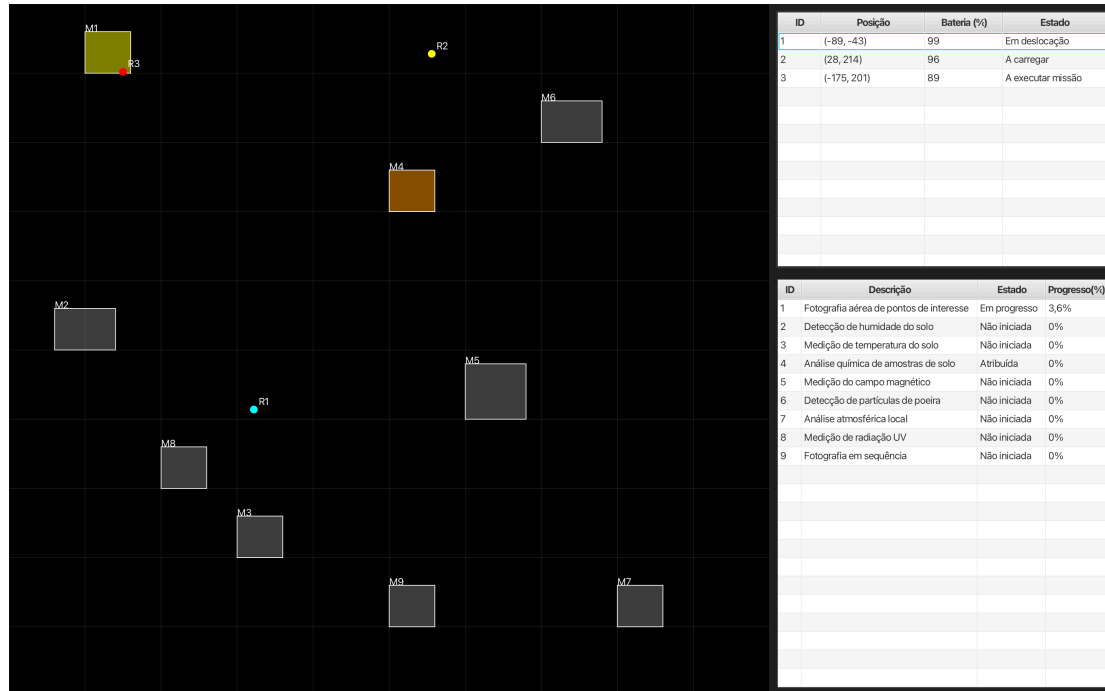


Figura 4: Interface Grafica GC

5.4. Ciclo de Atualização Gráfico

A interface é atualizada automaticamente a cada segundo, seguindo os seguintes passos:

1. Enviar GET /rovers;
2. Enviar GET /missoes;
3. Atualizar a posição do rover no mapa;
4. Recolorir a missão atualmente atribuída;
5. Redesenhar todas as missões nas coordenadas corretas;
6. Atualizar as tabelas laterais.

Este mecanismo permite ao operador observar:

- o movimento do rover em tempo real;
- a evolução do progresso das missões;
- alterações de estado (iniciada, em execução, concluída);
- autonomia do rover e eventuais anomalias.

5.5. Demonstração do Funcionamento

Durante a execução, o mapa mostra o rover a deslocar-se para a zona da missão atribuída, seguindo o planeamento definido no MissionLink. As missões permanecem visíveis no mapa, facilitando a visualização espacial da operação.

Quando uma missão muda de estado (por exemplo, “Atribuída” → “Em progresso”), a tabela correspondente atualiza-se imediatamente. O mapa permite confirmar visualmente a coerência entre a trajetória do rover e os dados reportados via TelemetryStream.

Esta visualização facilita:

- validação dos protocolos,
- deteção de comportamentos incorretos,
- análise das operações em ambiente realista.

Quando uma missão muda de estado a sua cor também muda para se perceber bem a diferença dos estados das missões.

O rover também muda de cor de acordo com o seu estado.

5.6. Justificação das Escolhas de Implementação

A implementação baseada numa API HTTP foi escolhida para:

- garantir independência total relativamente aos protocolos internos MissionLink (UDP) e TelemetryStream (TCP);
- permitir integração simples com aplicações externas (como o GC);
- manter o GC leve e desacoplado da lógica operacional da Nave-Mãe;
- utilizar apenas ferramentas nativas de Java (URLConnection), evitando dependências desnecessárias;
- suportar atualizações periódicas de forma simples, através de um ciclo com intervalo fixo de 1 segundo.

6. Testes e Resultados

Com o objetivo de avaliar a robustez do sistema de comunicações desenvolvido, foram realizados três cenários de teste distintos, nos quais se aumentaram progressivamente os valores de delay e packet loss entre a Nave Mãe e os rovers.

Em todos os testes, a fiabilidade da transmissão foi analisada através dos ficheiros de log gerados pela Nave Mãe, recorrendo ao script stats.py, que permitiu contabilizar:

- mensagens UPDATE e FILE_PART recebidas;
- pacotes duplicados, resultantes de retransmissões;
- pacotes únicos efetivamente entregues;
- pacotes perdidos.

6.1. Metodologia de Teste

Cada teste foi executado utilizando uma topologia CORE distinta (TP2-testeN.xml), mantendo a mesma lógica de funcionamento da aplicação, mas alterando:

- os valores de delay;
- a percentagem de packet loss;
- os parâmetros de retransmissão (timeout_millis e max_tentativas).

Os testes foram desenhados de forma incremental, permitindo observar a degradação gradual do desempenho do sistema e identificar o seu limite funcional.

6.2. Teste 1

No Teste 1 foram aplicadas condições de rede moderadas com perdas reduzidas.

O protocolo neste primeiro teste estava configurado com um timeout de 6s e um máximo de 5 retransmissões.

As condições de rede configuradas foram as seguintes:

- ligação entre a Nave Mãe (NM) e o router S1: delay de 1s e 5% de packet loss;
- ligação entre o Rover 1 e o router S2: delay de 2ms e 5% de packet loss;
- ligação entre o Rover 2 e o router S3: delay de 1s e 10% de packet loss;
- ligação entre o Rover 3 e o router S4: delay de 2s e 15% de packet loss.

Os resultados obtidos foram os seguintes:

Rover1:

UPDATES:
- Total recebidos: 49
- Repetidos: 5
- Updates únicos: 44
- Último UPDATE: 44
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 49

FILE_PARTS:
- Total recebidos: 16
- Repetidos: 0
- FILE_PART únicos: 16
- Último FILE_PART: 15
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 16

Rover2:

UPDATES:
- Total recebidos: 49
- Repetidos: 6
- Updates únicos: 43
- Último UPDATE: 43
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 49

FILE_PARTS:
- Total recebidos: 17
- Repetidos: 1
- FILE_PART únicos: 16
- Último FILE_PART: 15
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 17

Rover3:

UPDATES:
- Total recebidos: 33
- Repetidos: 4
- Updates únicos: 29
- Último UPDATE: 29
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 33

FILE_PARTS:
- Total recebidos: 22
- Repetidos: 5
- FILE_PART únicos: 17
- Último FILE_PART: 16
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 22

- todos os rovers receberam a totalidade dos UPDATEs esperados, sem perdas;
- o número de retransmissões foi reduzido;
- todas as partes dos ficheiros finais (FILE_PART) foram recebidas corretamente;
- não se registaram pacotes perdidos.

Este teste demonstra que o sistema funciona de forma fiável em condições de rede pouco adversas, apresentando apenas retransmissões pontuais.

6.3. Teste 2

No Teste 2 foram aplicadas condições de rede significativamente mais adversas, aumentando tanto os valores de delay como a taxa de packet loss, com o objetivo de avaliar o comportamento do sistema em cenários de comunicação instável.

O protocolo neste teste estava configurado com um timeout de 6s e um máximo de 5 retransmissões. De notar que uma vez que o RTT entre o Rover3 e a NM seria 8s, este timeout claramente não seria suficiente, o que deve levar a um elevado número de retransmissões, tal como foi observado.

As condições de rede configuradas foram as seguintes:

- ligação entre a Nave Mãe (NM) e o router S1: delay de 1s e 15% de packet loss;
- ligação entre o Rover 1 e o router S2: delay de 200ms e 15% de packet loss;
- ligação entre o Rover 2 e o router S3: delay de 1.5s e 25% de packet loss;
- ligação entre o Rover 3 e o router S4: delay de 3s e 40% de packet loss.

Os resultados obtidos foram seguintes:

Rover1:

UPDATES:

- Total recebidos: 36
- Repetidos: 9
- Updates únicos: 27
- Último UPDATE: 27
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 36

FILE_PARTS:

- Total recebidos: 21
- Repetidos: 2
- FILE_PART únicos: 19
- Último FILE_PART: 19
- Perdidos: 1 -> [7]
- ACKs enviados (FILE_PART): 21

Rover2:

UPDATES:

- Total recebidos: 41
- Repetidos: 12
- Updates únicos: 29
- Último UPDATE: 29
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 41

FILE_PARTS:

- Total recebidos: 15
- Repetidos: 8
- FILE_PART únicos: 7
- Último FILE_PART: 6
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 15

Rover3:

UPDATES:

- Total recebidos: 34
- Repetidos: 18
- Updates únicos: 16
- Último UPDATE: 16
- Perdidos: 0 -> []
- ACKs enviados (UPDATE): 34

FILE_PARTS:

- Total recebidos: 7
- Repetidos: 3
- FILE_PART únicos: 4

- Último FILE_PART: 3
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 7

- todos os rovers conseguiram concluir o envio dos UPDATES, sem perdas definitivas;
- o número de UPDATES únicos recebidos diminuiu à medida que as condições de rede se agravaram;
- a transmissão dos ficheiros finais (FILE_PART) começou a apresentar falhas pontuais, com perda de partes em um Rover;
- os mecanismos de retransmissão permitiram recuperar a maioria dos pacotes, embora com impacto no tempo total de transmissão.

Este teste demonstra que o sistema mantém funcionalidade em condições de rede adversas, embora com uma degradação visível na eficiência da transmissão e aumento da sobrecarga causada por retransmissões.

6.4. Teste 3

No Teste 3 foram aplicadas condições de rede ainda mais adversas, com o objetivo de identificar os limites funcionais do protocolo de comunicação e avaliar a eficiência do sistema em cenários críticos.

O protocolo neste teste estava configurado com um timeout de 8,5s e um máximo de 8 retransmissões. Este aumento visou compensar os elevados valores de delay e a taxa de packet loss mais elevada, especialmente entre o Rover3 e a Nave Mãe.

As condições de rede configuradas foram as seguintes:

- ligação entre a Nave Mãe (NM) e o router S1: delay de 1s e 20% de packet loss;
- ligação entre o Rover 1 e o router S2: delay de 4ms e 20% de packet loss;
- ligação entre o Rover 2 e o router S3: delay de 2s e 30% de packet loss;
- ligação entre o Rover 3 e o router S4: delay de 3s e 45% de packet loss.

Os resultados obtidos foram os seguintes:

Rover1:

- UPDATES:
- Total recebidos: 29
 - Repetidos: 11
 - Updates únicos: 18
 - Último UPDATE: 18
 - Perdidos: 0 -> []
 - ACKs enviados (UPDATE): 29

- FILE_PARTS:
- Total recebidos: 7
 - Repetidos: 2
 - FILE_PART únicos: 5
 - Último FILE_PART: 4
 - Perdidos: 0 -> []
 - ACKs enviados (FILE_PART): 7

Rover2:

- UPDATES:
- Total recebidos: 23
 - Repetidos: 9
 - Updates únicos: 14
 - Último UPDATE: 13
 - Perdidos: 0 -> []
 - ACKs enviados (UPDATE): 23

FILE_PARTS:

- Total recebidos: 7
- Repetidos: 1
- FILE_PART únicos: 6
- Último FILE_PART: 5
- Perdidos: 0 -> []
- ACKs enviados (FILE_PART): 7

Rover3:

UPDATES:

- Total recebidos: 19
- Repetidos: 10
- Updates únicos: 9
- Último UPDATE: 10
- Perdidos: 1 -> [7]
- ACKs enviados (UPDATE): 19

FILE_PARTS:

- Total recebidos: 24
- Repetidos: 12
- FILE_PART únicos: 12
- Último FILE_PART: 12
- Perdidos: 1 -> [3]
- ACKs enviados (FILE_PART): 24

- Rover1 conseguiu enviar todos os UPDATES e FILE_PARTs, embora com número elevado de retransmissões e redução do número de pacotes únicos recebidos;
- Rover2 apresentou redução significativa nos UPDATES únicos e nos FILE_PARTs enviados, mas conseguiu concluir a transmissão sem perdas definitivas;
- Rover3 registou perdas em UPDATES e em FILE_PARTs (1 pacote perdido de cada tipo), demonstrando os limites do protocolo em condições extremas.

O aumento de timeout e do número máximo de tentativas permitiu recuperar parte dos pacotes perdidos, mas não eliminou completamente a falha de transmissão em cenários críticos.

Este teste demonstra que o sistema ainda mantém funcionalidade sob condições de rede muito adversas, mas evidencia claramente os limites da eficiência do protocolo e o impacto da retransmissão sobre o tempo total de transmissão.

6.5. Discussão dos Resultados

Através dos três testes realizados, foi possível observar como a eficiência da transmissão se degrada à medida que as condições de rede se tornam mais adversas. Os principais pontos evidenciados são:

- O protocolo de retransmissão implementado mostrou-se eficaz na recuperação de pacotes perdidos, garantindo a entrega da maioria dos UPDATES e FILE_PARTs.
- O aumento do timeout e do número máximo de tentativas é crucial para manter a fiabilidade em cenários com maiores atrasos e perdas de pacotes.
- Existe um limite funcional do protocolo: o Rover3, nos cenários mais extremos (Teste 3), registou perdas tanto em UPDATES como em FILE_PARTs, evidenciando que, além de certo ponto, nem o aumento de tentativas é suficiente para compensar a degradação da rede.
- A eficiência de transmissão, isto é, o número de pacotes únicos recebidos em relação ao número de retransmissões, diminui à medida que a adversidade da rede aumenta.

Esta análise demonstra que o sistema é robusto e tolerante a falhas, mas também indica os limites práticos da solução face a condições de rede extremas.

7. Conclusão

A implementação desenvolvida ao longo deste trabalho permitiu construir um sistema funcional capaz de simular a comunicação entre uma Nave-Mãe e vários rovers, integrando dois protocolos aplicativos distintos e uma camada adicional de observação. O principal objetivo compreender, implementar e testar mecanismos de comunicação fiáveis sobre diferentes serviços de transporte, foi cumprido tanto através do MissionLink (UDP), onde a fiabilidade teve de ser garantida ao nível da aplicação, como através do TelemetryStream (TCP), que assegura a transmissão contínua da telemetria.

No caso do MissionLink, o maior desafio foi implementar um conjunto de mecanismos que compensassem as limitações naturais do UDP: confirmações explícitas, retransmissão, deteção de duplicados, números de sequência e controlo baseado em stop-and-wait. A utilização de handlers dedicados por rover, com filas separadas para pacotes e ACKs, revelou-se fundamental para garantir concorrência segura e isolamento entre as diferentes comunicações.

O TelemetryStream, por sua vez, permitiu explorar uma abordagem mais direta, utilizando TCP para proporcionar entrega fiável e ordenada da telemetria. A decisão de adotar um formato binário fixo, acompanhado do envio explícito do tamanho do pacote, simplificou a reconstrução dos dados e tornou o sistema eficiente mesmo com vários rovers a enviar dados simultaneamente.

A criação da API de Observação e do módulo Ground Control demonstrou como os dados produzidos pelos protocolos podem ser integrados e apresentados de forma estruturada. Esta camada de visualização foi importante para validar a coerência do sistema, permitindo observar o estado das missões e a mobilidade dos rovers de forma clara e acessível.

Os testes realizados permitiram validar o comportamento do sistema sob diferentes condições de rede, evidenciando tanto a sua robustez como os seus limites. Em cenários moderados, o MissionLink garantiu a entrega correta de UPDATES e FILE_PARTs com um número reduzido de retransmissões. Com o aumento da latência e da taxa de perda de pacotes, observou-se uma degradação progressiva da eficiência, refletida no aumento de pacotes duplicados e no impacto no tempo de transmissão. Nos cenários mais extremos, foram identificados os limites funcionais do protocolo, com perdas pontuais que não puderam ser totalmente compensadas apenas pelo aumento do timeout e do número máximo de tentativas.

De forma global, o trabalho permitiu aplicar vários conceitos abordados na unidade curricular, desde mecanismos de fiabilidade e concorrência até modelos de comunicação cliente-servidor e desenho de APIs. Embora existam melhorias possíveis, como otimização de tempos de retransmissão ou ampliação das funcionalidades do Ground Control, a solução desenvolvida cumpre os requisitos essenciais do enunciado e representa uma implementação sólida e funcional dos protocolos aplicativos propostos.